Process

Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Inter-process Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems

Process Concept

Process - a program in execution

More than a program(a text section)

Program is a **passive** entity where as process is an **active** entity.

Multiple parts

- The program code, also called **text section**
- Current activity including **program counter**, processor registers
- **Stack** containing temporary data
  - Function parameters, return addresses, local variables
- **Data section** containing global variables
- **Heap** containing memory dynamically allocated during run time

Process Concept…

Program is **passive** entity stored on disk (**executable file**); process is **active**

- Program becomes process when an executable file is loaded into memory

Execution of program started via GUI mouse clicks, command line entry of its name, etc.

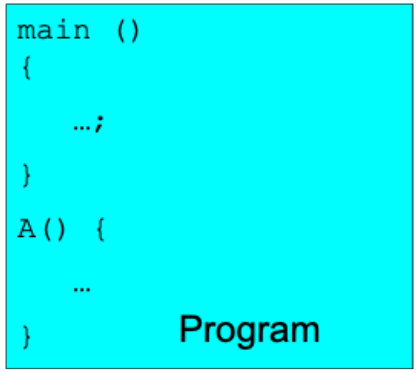
More to a process than just a program:

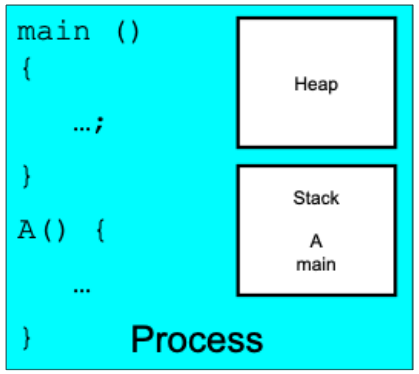
Program is just part of the process state

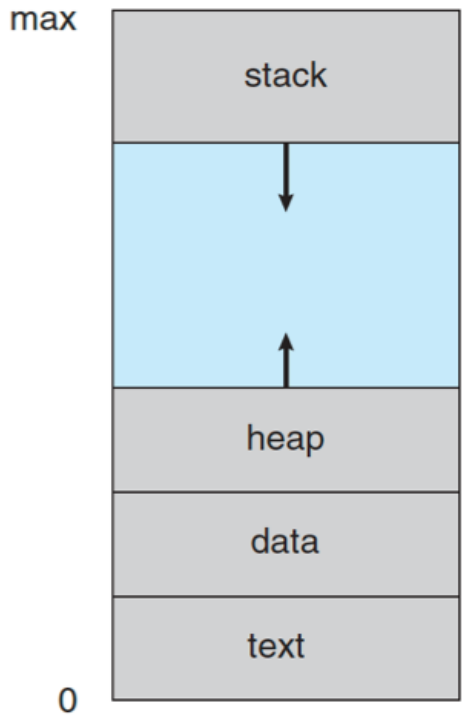
A program can invoke more than one process.

Ex: Many copies of the editor program, data sections vary.

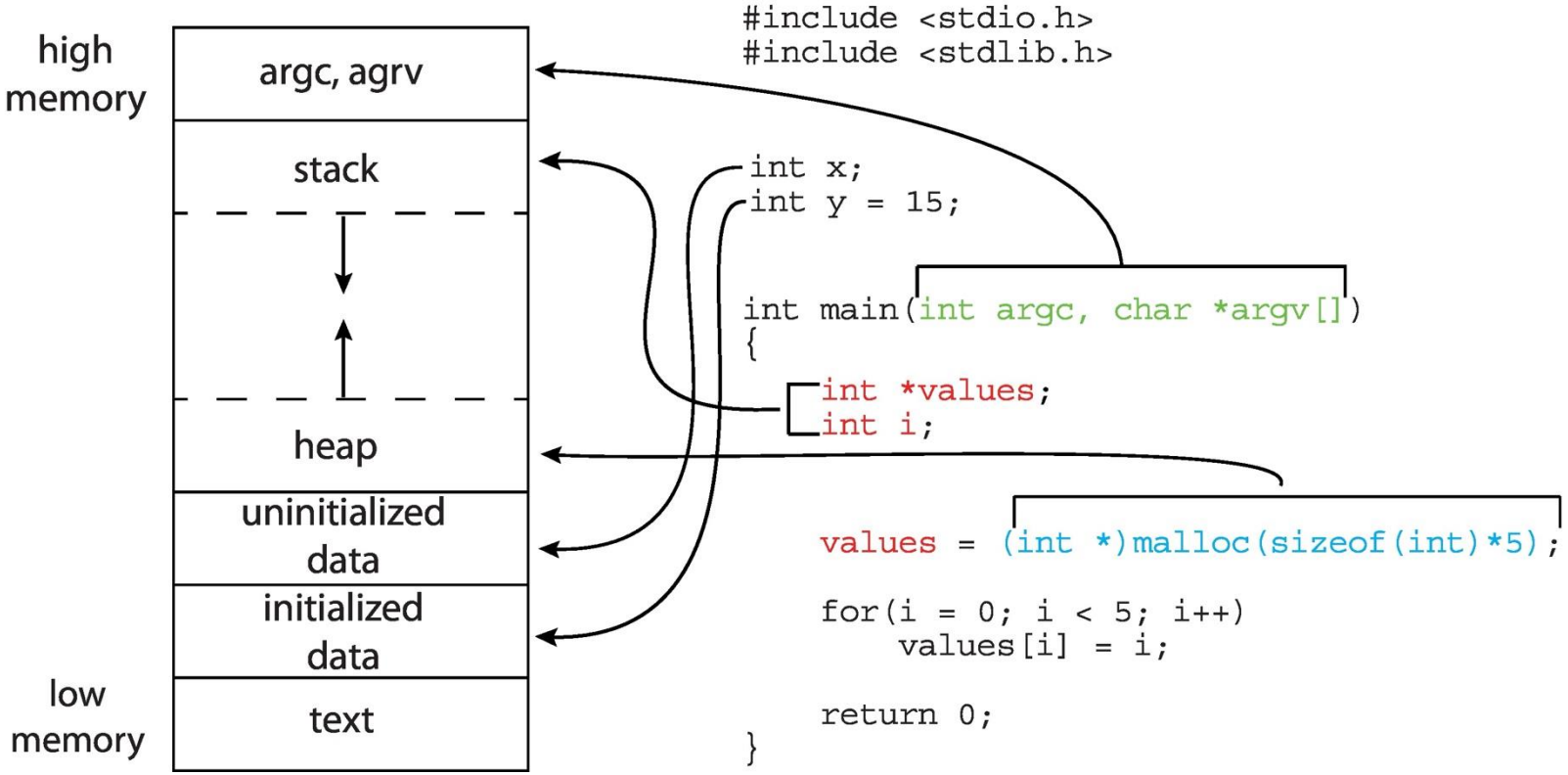
Process Concept…







Memory Layout of a C Program



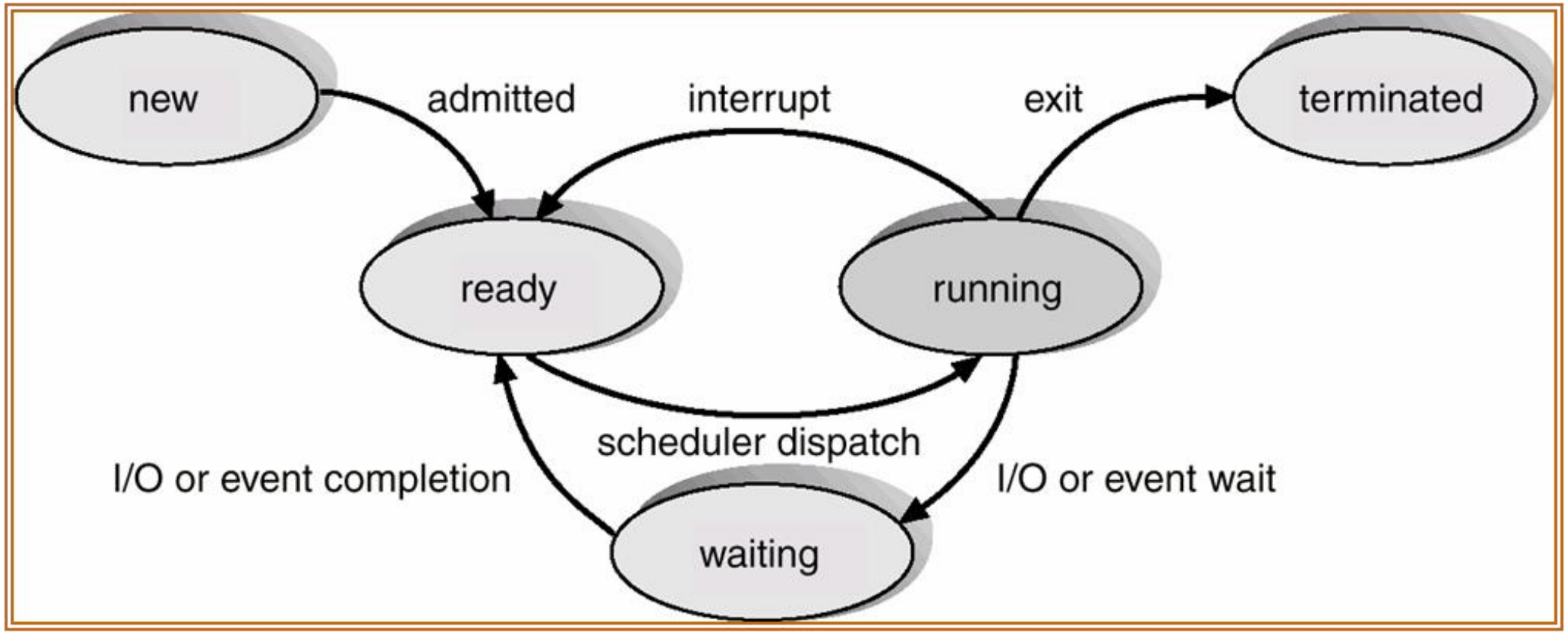
BSS (Block Started by Symbol)

Process State

As a process executes, it changes **state**

- **New**: The process is being created
- **Running**: Instructions are being executed
- **Waiting**: The process is waiting for some event to occur
- **Ready**: The process is waiting to be assigned to a processor
- **Terminated**: The process has finished execution

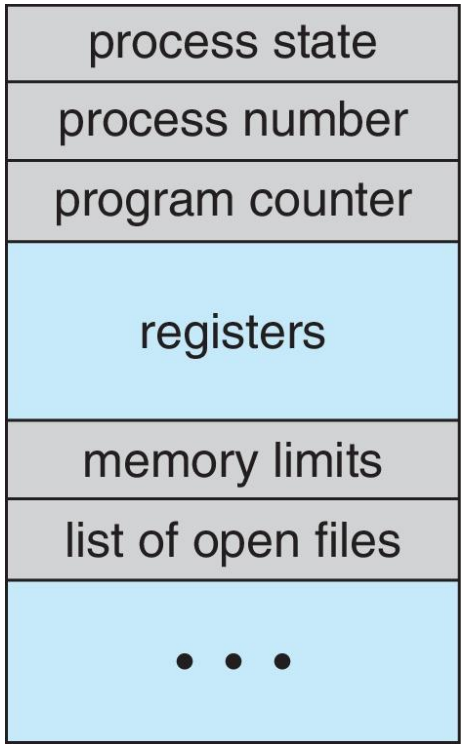
Process State Transitions



Process Control Block (PCB)

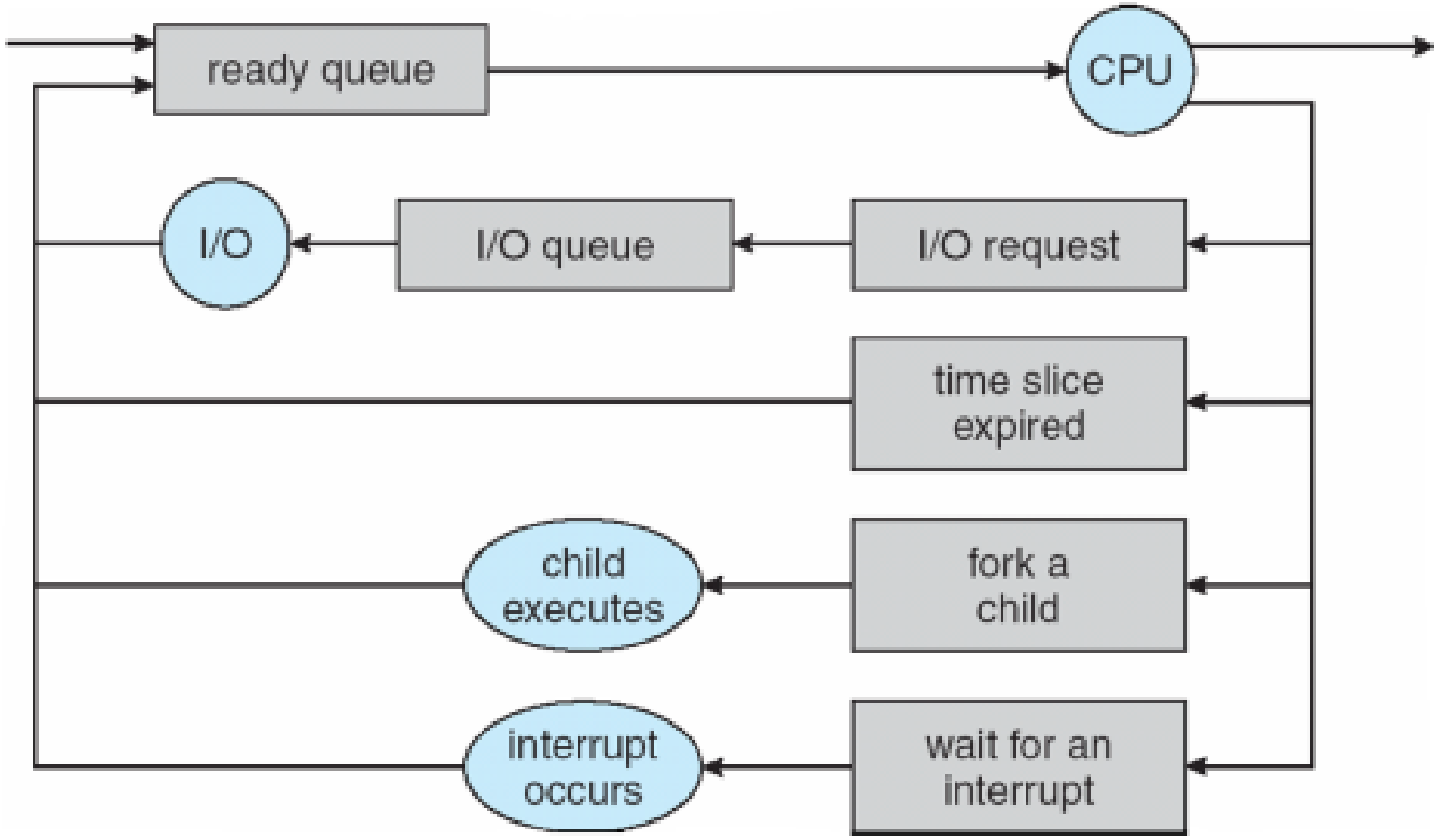
Information associated with each process(also called **task control block**)

- Process state – running, waiting, etc.
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

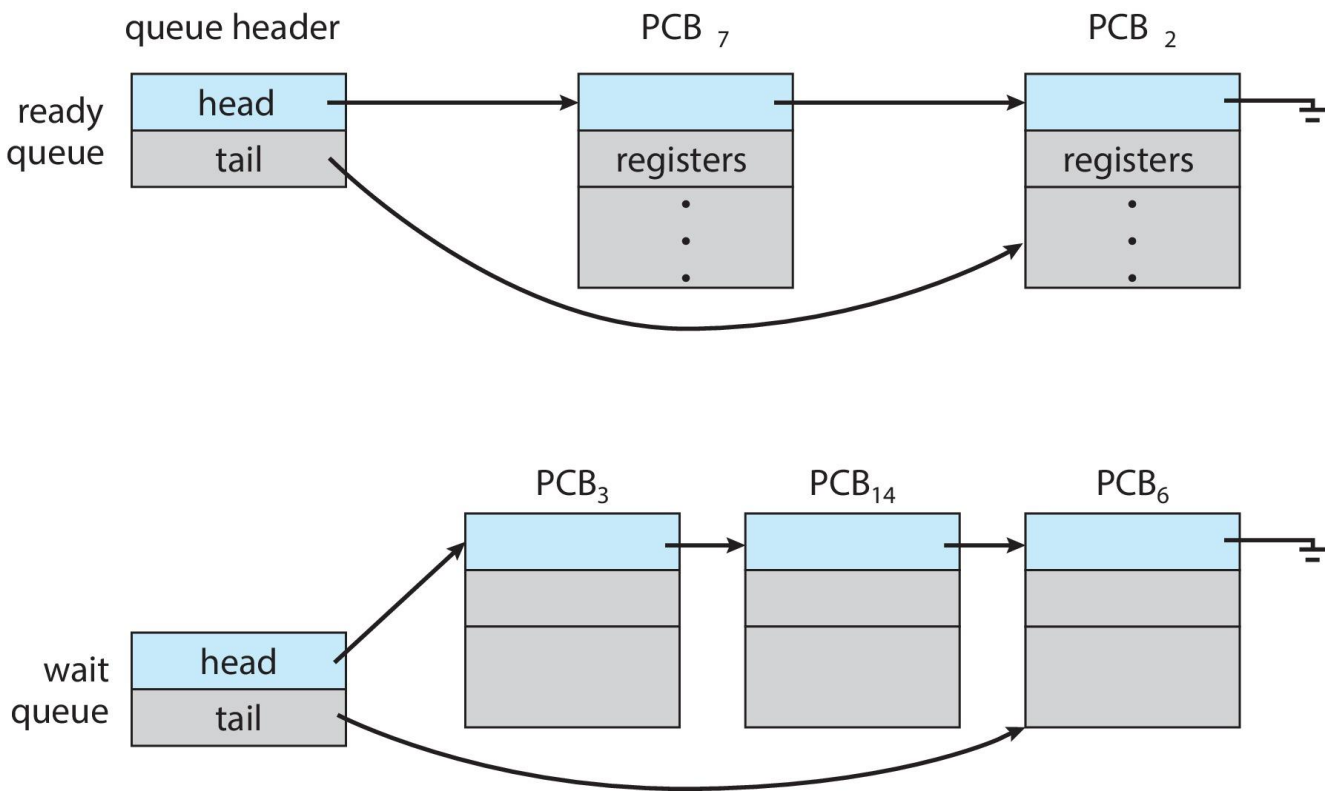


Process Schedulers

- Long term scheduler
- Short term scheduler
- Medium term scheduler
- Processes can be described a
  - I/O bound process
  - CPU bound process



Ready and Wait Queues



CPU Switch From Process to Process

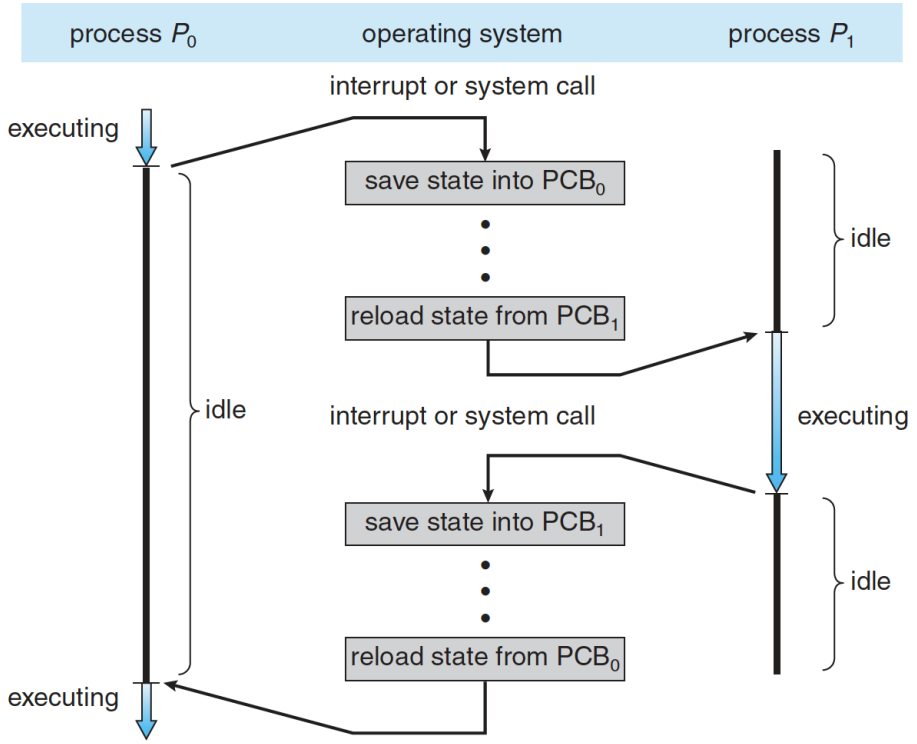


Figure 3.4 Diagram showing CPU switch from process to process.

Context Switch

When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**

Context of a process represented in the PCB

Context-switch time is pure overhead; the system does no useful work while switching

- The more complex the OS and the PCB ➔ the longer the context switch

Operations on Processes

System must provide mechanisms for:

- Process creation
- Process termination

Process Creation

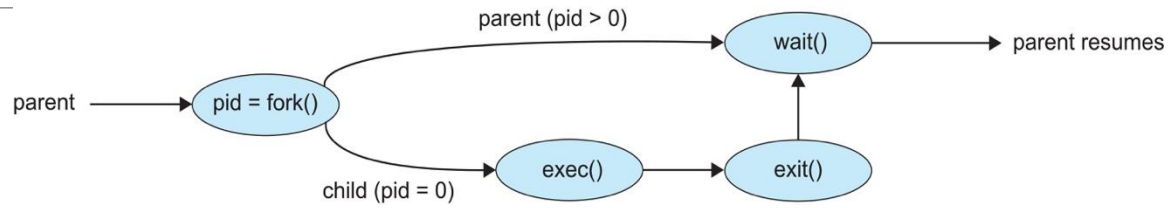
Parent process create **children** processes, which, in turn create other processes, forming a **tree** of processes

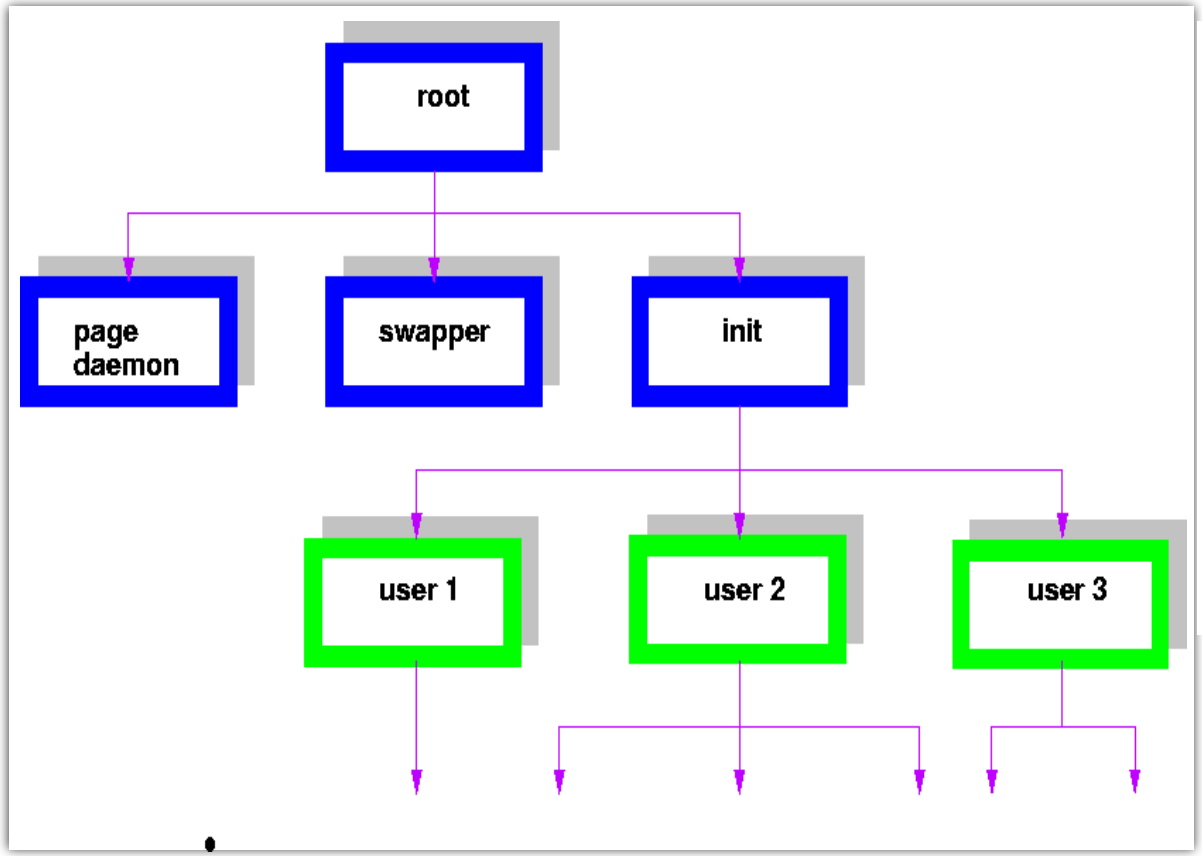
Generally, process identified and managed via a **process identifier** (**pid**)

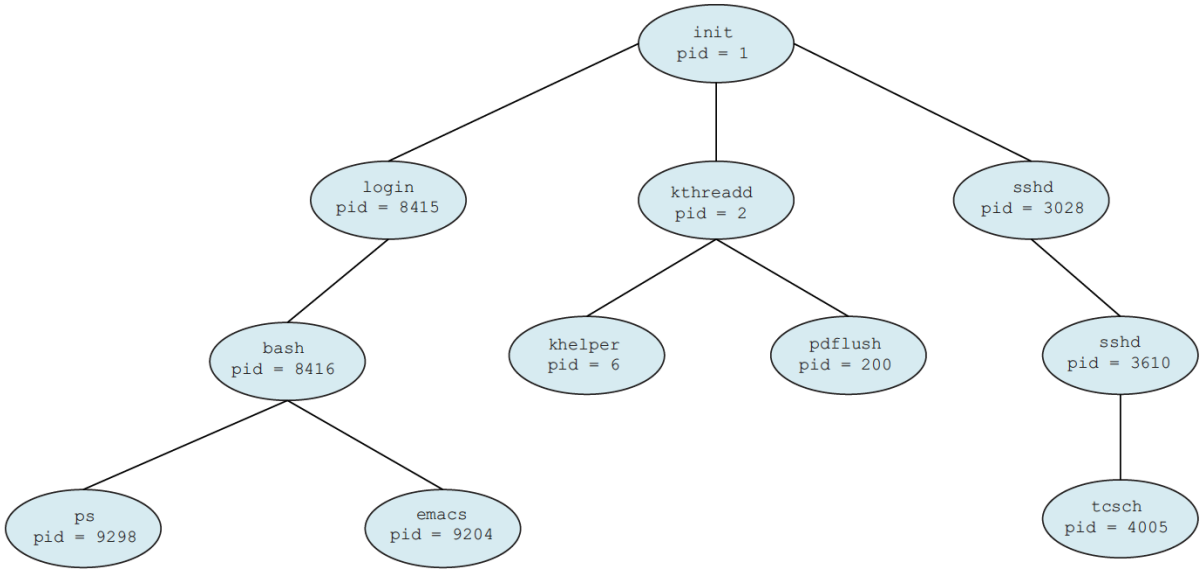
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate

Process Creation…

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - `fork()` system call creates new process
  - `exec()` system call used after a `fork()` to replace the process' memory space with a new program
  - Parent process calls `wait()` waiting for the child to terminate




root
page daemon
swapper
init
user 1
user 2
user 3


init
pid = 1
login
pid = 8415
kthreadd
pid = 2
sshd
pid = 3028
bash
pid = 8416
khelper
pid = 6
pdflush
pid = 200
sshd
pid = 3610
ps
pid = 9298
emacs
pid = 9204
tcsch
pid = 4005

C Program Forking Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

Process Termination

- Process executes last statement and then asks the operating system to delete it using the `exit()` system call.
  - Returns status data from child to parent (via `wait()`)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the `abort()` system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting, and the operating systems does not allow a child to continue if its parent terminates

Process Termination…

Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.

- **cascading termination.** All children, grandchildren, etc., are terminated.
- The termination is initiated by the operating system.

The parent process may wait for termination of a child process by using the `wait()` system call . The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

If no parent waiting (did not invoke `wait()`) process is a **zombie**

If parent terminated without invoking `wait()`, process is an **orphan**

Interprocess communication

Processes within a system may be ***independent*** or ***cooperating***

Cooperating process can affect or be affected by other processes, including sharing data

Reasons for cooperating processes:

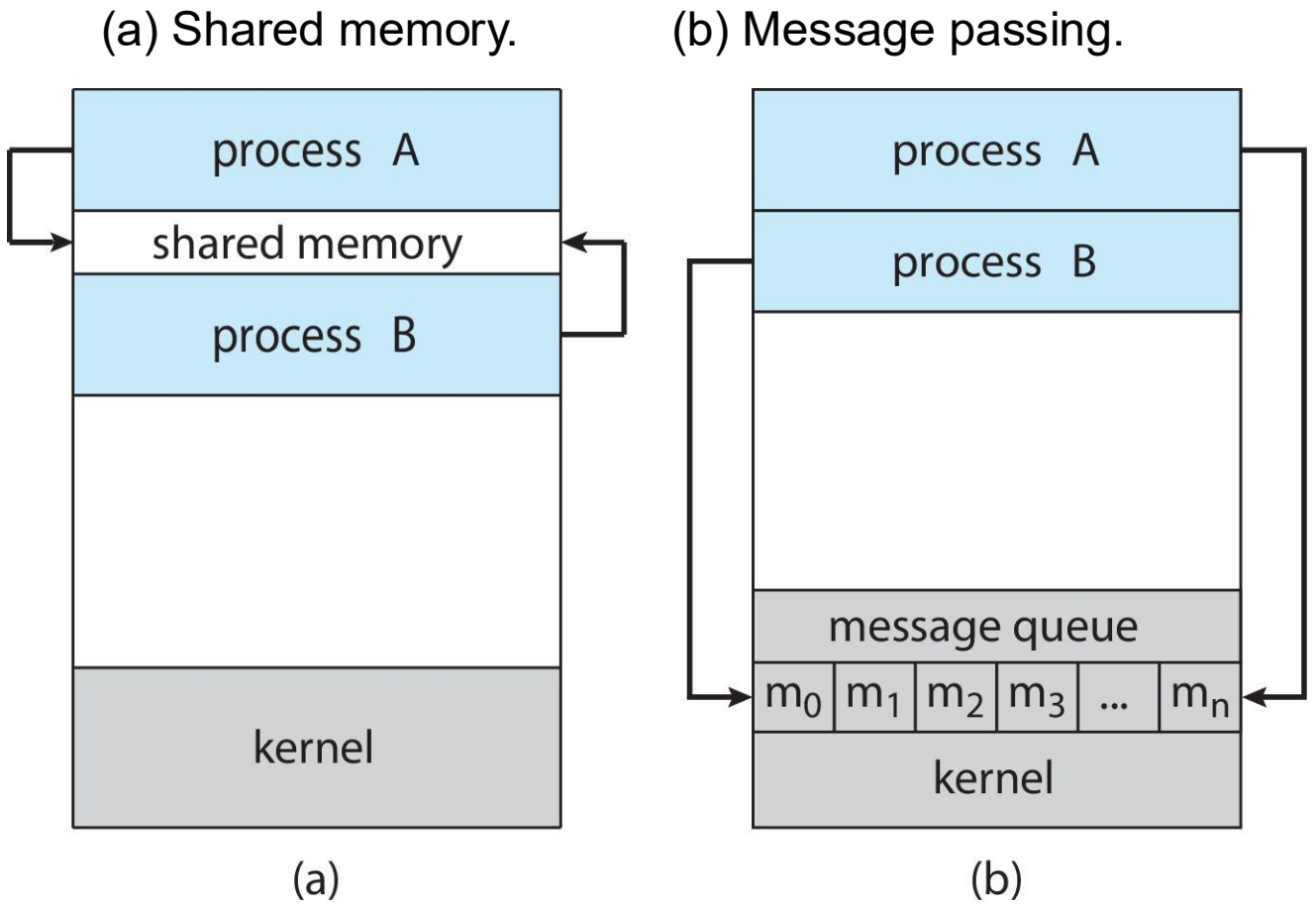
- Information sharing
- Computation speedup
- Modularity
- Convenience

Cooperating processes need **interprocess communication** (**IPC**)

Two models of IPC

- **Shared memory**
- **Message passing**

Communications Models



IPC – Shared Memory

- A region of memory that is shared by cooperating process is established.
- Resides in the address space of the process creating the shared memory segment.
- Process that wish to communicate may attach it to their address space.
- Processes must agree to remove the memory restrictions.
- Form of data & location are determined sharing processes. OS is not involved.
- Processes take care of synchronization

IPC – Message Passing

- Provides a mechanism to allow process to communicate and synchronize without sharing the same address space.
  - Useful in distributed environment. ex chat on internet.
- IPC facility provides two operations
  - send(message) - message size fixed/variable
  - receive(message)
- If processes P and Q wish to communicate, they need to:
  - establish a communication link between them
  - exchange messages via send/receive

Message Passing ...

If processes *P* and *Q* wish to communicate, they need to:

- Establish a ***communication link*** between them
- Exchange messages via send/receive

Implementation issues:

- How are links established?
- Can a link be associated with more than two processes?
- How many links can there be between every pair of communicating processes?
- What is the capacity of a link?
- Is the size of a message that the link can accommodate fixed or variable?
- Is a link unidirectional or bi-directional?

IPC – Message Passing…

- Fixed vs. Variable size message
  - Fixed message size - straightforward physical implementation, programming task is difficult.
  - Variable message size - simpler for programming, more complex physical implementation.

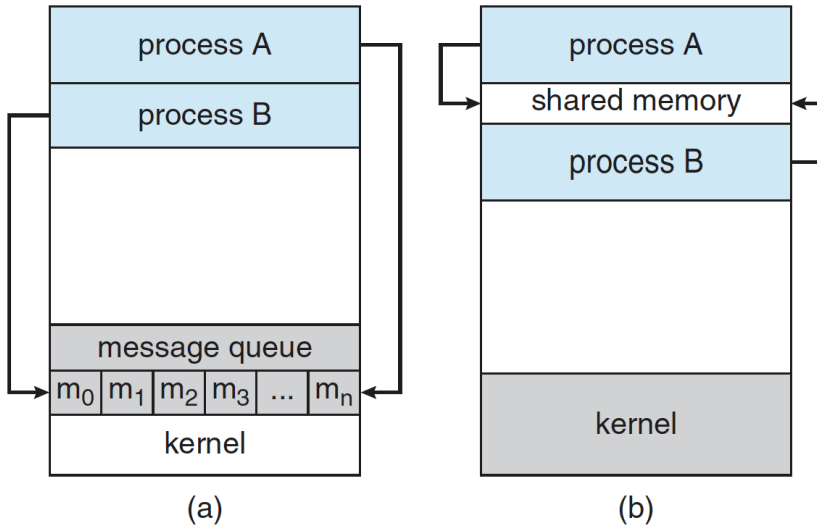


Figure 3.12 Communications models. (a) Message passing. (b) Shared mer

Implementation of Communication Link

- Physical:
  - Shared memory
  - Hardware bus
  - Network
- Logical:
  - Direct or indirect
  - Synchronous or asynchronous
  - Automatic or explicit buffering

Message Passing...

Methods for logically implementing the link and send () / receive() operations

- Direct or indirect communication
- Synchronous or asynchronous communication
- Automatic or explicit buffering.

- Direct communication
  - Sender and Receiver processes must name each other explicitly:
    - **send**(*P*, *message*) - send a message to process P
    - **receive**(Q, message) - receive a message from process Q
  - Properties of communication link:
    - Links are established automatically.
    - A link is associated with exactly one pair of communicating processes.
    - Exactly one link between each pair
  - Symmetry and Asymmetry

Message Passing...

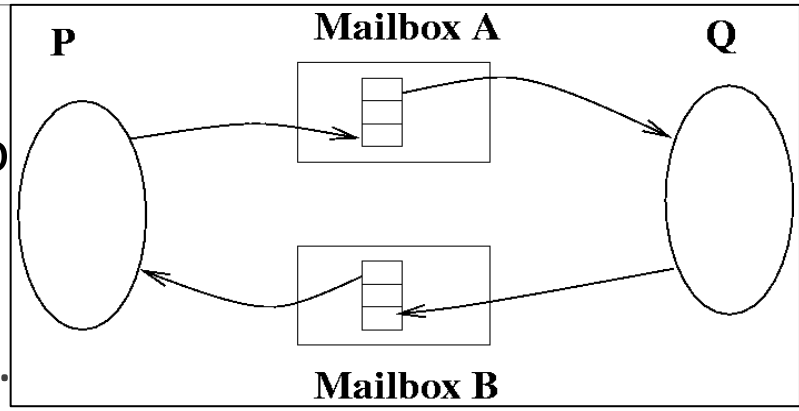
- Indirect Communication

  Messages are directed to and received from mailboxes (also called ports)

  - Unique ID for every mailbox.
  - Processes can communicate only if they share a mailbox.

    Send(*A*, *message*) /* send *message* to mailbox *A* */

    Receive(*A*, *message*) /* receive *message* from mailbox *A* */



  Properties of communication link

  - Link established only if processes share a common mailbox.
  - Link can be associated with many processes.
  - Pair of processes may share several communication links

Message Passing...

- Indirect Communication…

  Operations

  - create a new mailbox
  - send/receive messages through mailbox
  - destroy a mailbox

  Mailbox can be owned by process or OS. If its owned by process then we distinguish between owner and user. No issues arise.

  If its owned by OS then Issue may arise

  - P1, P2 and P3 share mailbox A.
  - P1 sends message, P2 and P3 receive… who gets message??

  Possible Solutions

  - disallow links between more than 2 processes
  - allow only one process at a time to execute receive operation
  - allow system to arbitrarily select receiver and then notify sender.

Message Passing…

- Synchronization
  - Message passing may be either blocking or non-blocking
  - Blocking is considered synchronous
    - *Blocking send* the sender blocked until the message is received
    - *Blocking receive* the receiver is blocked until a message is available
  - Non-blocking is considered asynchronous
    - *Non-blocking send* has the sender send the message and continue
    - *Non-blocking receive* has the receiver receive a valid message or null

Message Passing...

- Buffering

Link has some capacity - determine the number of messages that can reside temporarily in it.

- *Zero-capacity Queues*: 0 messages - sender blocks until recipient receives the message.
- *Bounded capacity Queues*: Finite length of *n* messages - If the queue is not full when a new message is sent, the message is placed in the queue. Sender can continue without waiting. If the link is full then the sender must block until the space is available in the queue.
- *Unbounded capacity Queues*: Infinite queue length - sender never blocks.

Producer-Consumer: Message Passing

Producer

```
message next_produced;
while (true) {
   /* produce an item in next_produced */

   send(next_produced);
  }
```

Consumer

```
message next_consumed;
while (true) {
    receive(next_consumed)

   /* consume the item in next_consumed */
}
```

Examples of IPC Systems - POSIX

POSIX Shared Memory

- Process first creates shared memory segment
  `shm_fd = shm_open(name, O CREAT | O RDWR, 0666);`
- Also used to open an existing segment
- Set the size of the object

```
ftruncate(shm_fd, 4096);
```

- Use `mmap()` to memory-map a file pointer to the shared memory object
- Reading and writing to shared memory is done by using the pointer returned by `mmap()`.

IPC POSIX Producer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* strings written to shared memory */
const char *message_0 = "Hello";
const char *message_1 = "World!";

/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* write to the shared memory object */
    sprintf(ptr,"%s",message_0);
    ptr += strlen(message_0);
    sprintf(ptr,"%s",message_1);
    ptr += strlen(message_1);

    return 0;
}
```

IPC POSIX Consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s",(char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```

Pipes

Acts as a conduit allowing two processes to communicate

Issues:

- Is communication unidirectional or bidirectional?
- In the case of two-way communication, is it half or full-duplex?
- Must there exist a relationship (i.e., ***parent-child***) between the communicating processes?
- Can the pipes be used over a network?

Ordinary pipes – cannot be accessed from outside the process that created it. Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.

Named pipes – can be accessed without a parent-child relationship.

Ordinary Pipes

Ordinary Pipes allow communication in standard producer-consumer style

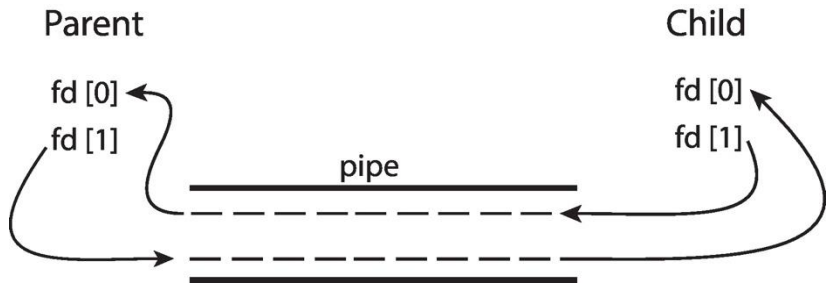
Producer writes to one end (the **write-end** of the pipe)

Consumer reads from the other end (the **read-end** of the pipe)

Ordinary pipes are therefore unidirectional

Require parent-child relationship between communicating processes

Windows calls these **anonymous pipes**



Named Pipes

- Named Pipes are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication
- Provided on both UNIX and Windows systems